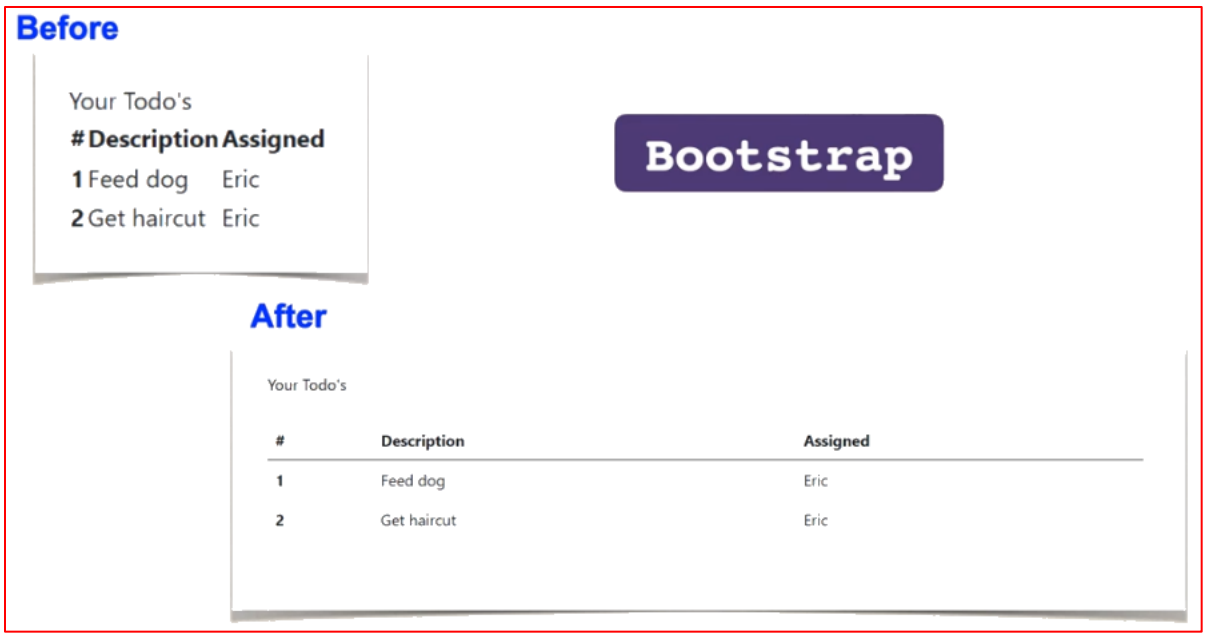
Adding CSS to a React Project

The next step in our course is to enhance our application's aesthetics by integrating React with Bootstrap. This will transform our rather rudimentary layout into a visually appealing interface.

Currently, our application's todo list is rather squeezed together, lacking any style. It simply displays our tasks as "1. Feed dog, Eric" and "2. Get a haircut, Eric," without any separation or emphasis. Understandably, this doesn't make for an engaging user experience.

But once we implement Bootstrap, a remarkable transformation occurs. We'll be able to display the tasks in a clean, organized table format, making it far more attractive to the user.



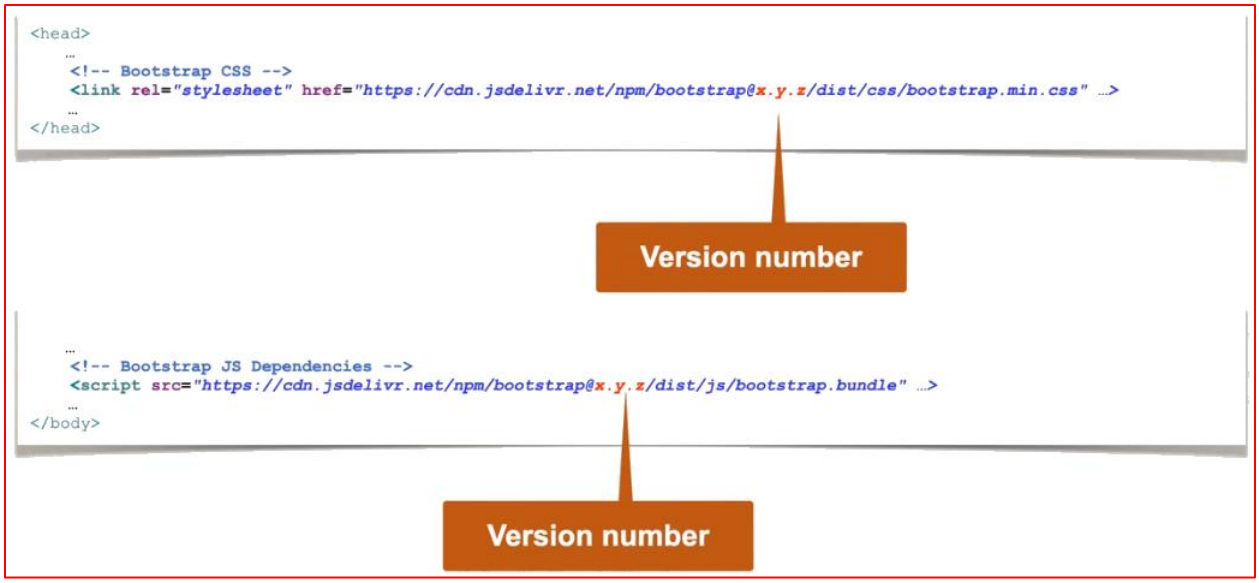
So, how are we going to achieve this transformation? Here's a step-by-step guide to the development process:

Step 1: Obtain Links for Remote Bootstrap Files

We must first visit the Bootstrap website, found at `www.getbootstrap.com`. The website provides instructions on how to get started. We'll navigate there to grab the necessary links to import into our application.

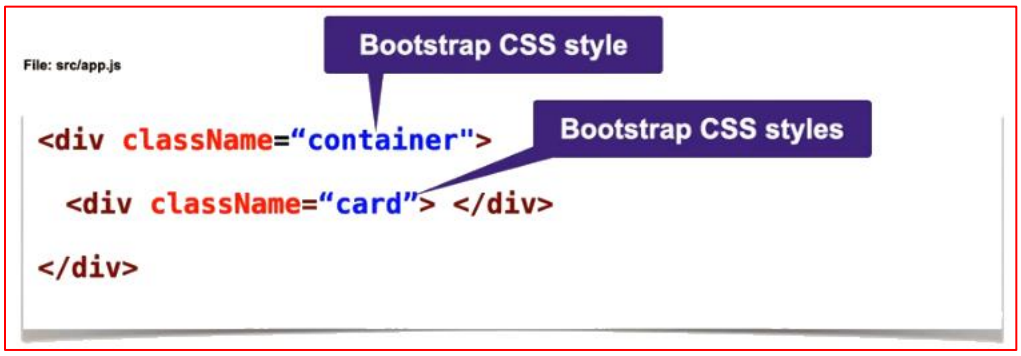
Step 2: Add Links to `index.html`

This step involves embedding the Bootstrap CSS and JavaScript dependencies into our `index.html` file. You'll notice attributes like `stylesheet`, and the reference URL will contain an XYZ, which is the specific version number of Bootstrap we'll be implementing.



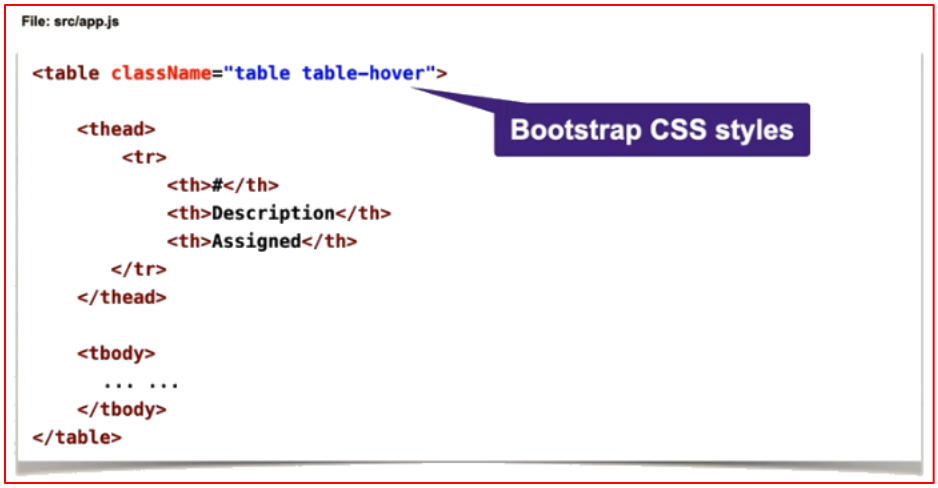
Step 3: Apply Bootstrap CSS Styles to Component HTML Template

With Bootstrap now linked to our project, we'll begin applying CSS styles to our component's HTML template. You'll notice class names like `container` and `card`. These are specific to Bootstrap and will begin to transform our React application, lending it a more refined appearance.



Step 4: Apply Bootstrap CSS Styles to Component HTML Table

The final step involves applying Bootstrap CSS styles to our table inside the `App.js` file. Class names such as `table` and `table-hover` are essential here. By integrating these CSS classes throughout our application, we can create a responsive, elegant interface.



Now, our once mundane todo application will become an engaging tool, with a pleasing user experience and an intuitive user interface.

In summary, Bootstrap will enable us to present the same information - your todos, descriptions, and assignments - in a visually stunning table format. This will not only enhance the appearance but also improve usability.

Stay tuned, as we'll jump into the practical steps and add Bootstrap to our project!

Add CSS to our React Project

Let’s inject some style into our application by adding CSS. Specifically, we'll be working with Bootstrap, a renowned library for designing sleek and responsive layouts. Our goal is to transform the bland appearance of our todo list into something significantly more appealing.

Let's break down the steps.

Using Bootstrap Version 5

For the demonstration, we'll be using Bootstrap version 5. If you'd like to follow along precisely, please ensure that you're using this version. Bootstrap relies on both CSS and JavaScript, and we'll be incorporating both into our project.

Adding Bootstrap to the Project

Scrolling through the Bootstrap documentation (https://getbootstrap.com/docs/5.0/getting-started/introduction/) , you'll find details about the CSS and JavaScript that make up the library, including the starter template. From there, you can copy the link to the Bootstrap CSS file.




```
<!-- Bootstrap CSS -->
    <link
href="https://cdn.jsdelivr.net/npm/bootstrap@5.0.2/dist/css/bootstrap.min.css
" rel="stylesheet" integrity="sha384-
EVSTQN3/azprG1Anm3QDgpJLIm9Nao0Yz1ztcQTwFspd3yD65VohhpuuCOmLASjC"
crossorigin="anonymous">
```

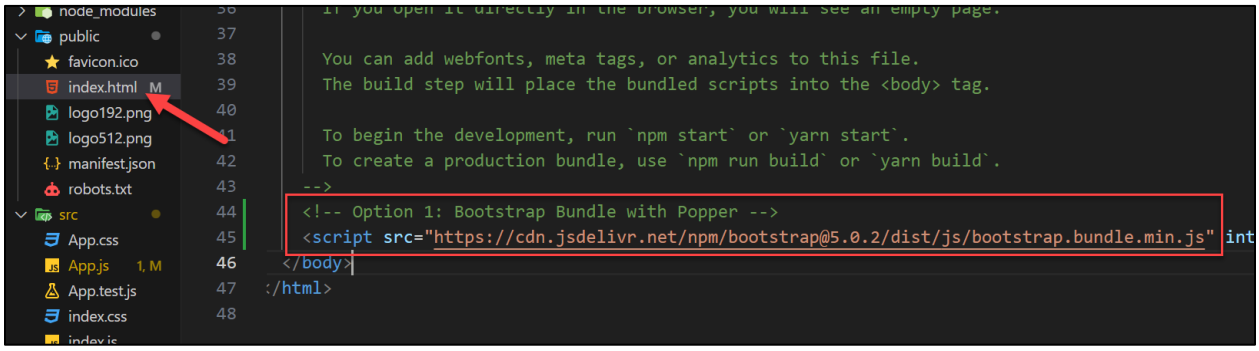
After copying it, head back to your application and navigate to your `index.html` file. Scroll down to find the manifest, and paste the link as the last link there.



Next, find the option for the Bootstrap bundle with Popper in the Bootstrap documentation, and copy all of that. Paste this just above the closing body tag in your application.




```
<!-- Option 1: Bootstrap Bundle with Popper -->
    <script
src="https://cdn.jsdelivr.net/npm/bootstrap@5.0.2/dist/js/bootstrap.bundle.mi
n.js" integrity="sha384-
MrcW6ZMFYlzcLA8Nl+NtUVF0sA7MsXsP1UyJoMp4YLEuNSfAP+JcXn/tWtIaxVXM"
crossorigin="anonymous"></script>
```



Once completed, your application's appearance won't change yet, but you'll have successfully implemented Bootstrap's CSS and JavaScript into your project.

Implementing Bootstrap CSS

Now, let's add some life to our design. Start by opening your JavaScript file and locating the first `div`. Add `className="mt-5 container"`. This will make your table move up and down a bit. Here, `container` centers the content, and `mt-5` (margin-top 5) pushes it down.

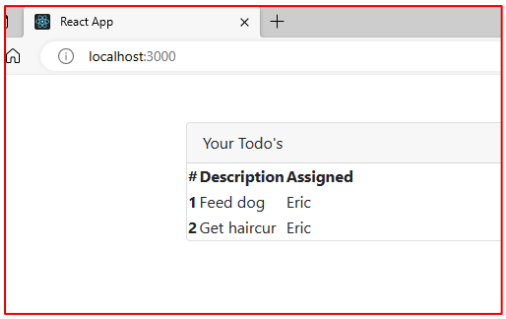
Note the use of `className` instead of `class`. While static HTML uses `class` to define CSS, React creates the DOM using JSX, which requires the `className` property.

Proceed with the following class names in the next `div`s:

- `class name="card"`
- `class name="card-header"`
- `class name="card-body"`

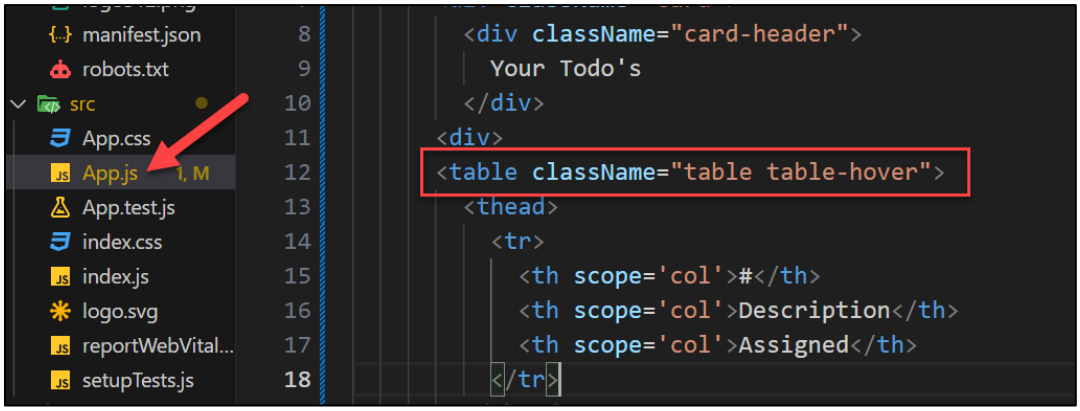


Once you save and refresh your application, you'll notice that the layout is starting to take shape. The Bootstrap classes have started to affect the appearance, adding structure without altering the underlying data.



Enhancing the Table

Now let's move to the table and add `className="table table-hover"`.



Return to your application, and you'll see that the design has further improved. Your todos are neatly organized, with a header and attractive table rows that respond to mouse hover. It's a transformation achieved with minimal effort thanks to Bootstrap.